Metaheuristics Algorithum for Machine Scheudling

基於啟發式演算法解決機器排程問題

1st Hung Ming-Chun
Department of Computer Science and Information Engineering
Nation Taiwan Normal University
Taipei, Taiwan(R.O.C)
61447026s@ntnu.edu.tw

2nd Hsieh Yao-yu
Graduate Institude of Science Education
Nation Taiwan Normal University
Taipei, Taiwan(R.O.C)
61345001s@ntnu.edu.tw

3rd Given Name Surname
Department of Computer Science and Information Engineering
Nation Taiwan Normal University
Taipei, Taiwan(R.O.C)
@ntnu.edu.tw

***Abstract*—**

I. Introduction

在傳統直接式演算法中，在處理 NP 問題時會遇到非常嚴重的複雜度瓶頸，而啟發式演算法可以透過迭代，能在固定時間下取得十分近似正確解答案，而該研究透過三種演算法分別是 Iterative Improvement、Simulated Annealing、Tabu Search，並且測試不同的退火參數以及 Tabu Tenure Object 來測試運行結果。

II. Related Work

A. Scheduling Problem

定序流線型工廠排程問題 (Permutation Flowshop Scheduling Problem, PFSP) 旨在決定 n 個工作在 m 台機器上的最佳加工順序。

本實驗之輸入包含工作集合 $J = \{j_1, j_2, \ldots, j_n\}$、機器集合 $M = \{M_1, M_2, \ldots, M_m\}$ 以及加工時間矩陣 $p_{i,j}$。

核心限制條件如下：所有工作在各台機器上的加工順序 π 必須完全一致，且機器在同一時間僅能處理單一工作，加工過程不可中斷。目標函數為極小化總完工時間 (Makespan)，記作 C_{max}。

定義 $C(\pi_k, i)$ 為位置 k 的工作在機器 i 的完工時間，其遞迴關係如下：

$$C(\pi_k, i) = \max\{C(\pi_{k-1}, i), C(\pi_k, i-1)\} + p_{i,\pi_k} \tag{1}$$

最終目標為尋求一排列 π^* 使得 $Z = C(\pi_n, m)$ 達最小值。

本實驗之演算法實作包含以下兩個核心要素：

- 解表示法 **(Representation)**：採工作序列編碼。若有 n 個工作，則解為一長度為 n 的排列向量。
- 鄰域函式 **(Neighborhood Function)**：採用交換移動 (Swap)。定義鄰域運算為隨機選取序列中兩個位置並交換其內容，以產生新的可行解。

III. Trajectory-based Metaheuristics Algorithum

本研究實作三種啟發式演算法以求解 PFSP 問題。各個演算法皆基於鄰域搜尋架構，透過不同的接受準則與搜尋策略，在解空間中尋找近似最佳解。以下詳述各演算法之實現細節。

A. Iterative Improvement

反覆改善法採貪婪策略。其接受條件 **(Acceptance Criterion)** 為：僅當鄰近解 π' 的目標值優於當前解 π 時，方接受更新。其邏輯如下式：

$$f(\pi') < f(\pi) \tag{2}$$

B. Simulated Annealing

模擬退火法引入機率機制以跳脫區域最佳解。其冷卻時程 **(Cooling Schedule)** 採用幾何降溫模型，相關數學關係與參數如下：

$$T_{k+1} = \alpha \times T_k \tag{3}$$

$$P(\text{accept}) = \exp\left(\frac{-\Delta f}{T}\right) \tag{4}$$

其中變數 α 為冷卻率 (設定於第 III-B 節)，T 為當前溫度。當 $\Delta f > 0$(即新解較差) 時，仍有 P 之機率接受該解。

C. Tabu Search

禁忌搜尋法利用記憶機制避免重複搜尋。其禁忌列表 **(Tabu List)** 儲存的是最近執行的交換動作 (即工作對及其位置)，旨在防止搜尋過程回流至已探索區域。此外，本實作包含渴望條件 **(Aspiration Criterion)**：若某一受禁忌限制的移動能產生優於目前史上最佳解 f_{best} 的結果，則無視禁忌狀態強制接受該移動。

IV. Implement Detail

A. Scheduling Problem

本研究針對排列流水線排程問題（Permutation Flowshop Scheduling Problem）設計了 `Scheduling` 類別作為評估函數（Evaluator）。給定一組工作的排列順序（Order），系統會依序將工作排入機器中。假設共有 m 台機器，系統會維護一個大小為 m 的陣列 `machine_end_time` 來記錄每台機器的最早可用時間。

對於排列中的每一個工作，其在第 m_i 台機器的開始時間取決於「該工作在前一台機器的完工時間」與「第 m_i 台機器的最早可用時間」兩者中的最大值：

$$start_time = \max(current_job_ready_time, machine_end_time[m_i])$$

接著將 $start_time$ 加上該工作在該機器的處理時間 $job_mtx[m_i][job]$ 即為完工時間。依此邏輯計算至最後一台機器，最後一個工作的完工時間即為該排程的總完工時間（Makespan）。

B. Iterative Improvement

Iterative Improvement (II) 演算法在此實作中採用隨機爬山演算法（Stochastic Hill Climbing）的邏輯。演算法首先透過 `std::shuffle` 產生一組隨機排列作為初始解。在每一次迭代中，透過交換鄰域（Swap Neighborhood）隨機挑選兩個相異的索引位置並交換工作，產生一個相鄰解（Neighbor）。

若該相鄰解的 Makespan 嚴格小於當前解的 Makespan，則演算法會接受該相鄰解並將其設為新的當前解；若較差或相等則捨棄。此過程重複執行指定的迭代次數（`max_iter`），並記錄搜尋軌跡以供後續分析。

C. Simulated Annealing

模擬退火演算法（Simulated Annealing, SA）同樣採用交換鄰域隨機產生相鄰解。為了賦予演算法跳脫區域最佳解（Local Optima）的能力，SA 引入了 Metropolis 準則。

令 $\Delta = neighbor_makespan - current_makespan$。若 $\Delta < 0$（解獲得改善），則無條件接受該相鄰解；若 $\Delta \geq 0$（解變差），則以機率 P 決定是否接受該劣解：

$$P = \exp\left(-\frac{\Delta}{T}\right)$$

其中 T 為當前溫度。每次迭代結束後，溫度會依據幾何降溫排程進行衰減：$T_{new} = T_{old} \times cooling_rate$。演算法在過程中會持續追蹤並更新歷史最佳解（Global Best）。

D. Tabu Search

禁忌搜尋（Tabu Search, TS）採用最佳改善（Best Improvement）策略，在每一代中透過雙層迴圈走訪所有 $O(n^2)$ 個可能的交換組合，並挑選出非禁忌狀態下（或符合特赦條件）的最佳相鄰解。為了避免演算法陷入循環，本實作設計了兩種不同的禁忌名單（Tabu List）記錄機制：

- **Method 1:** 基於動作的禁忌記錄 **(Action-based Tabu List)**
  在 `tabu_search` 函數中，禁忌名單實作為一個大小為 $n \times n$ 的二維陣列。當工作 i 與工作 j 被交換後，系統會將這對組合寫入陣列中，記錄其「解禁代數」。為了統一索引並節省空間，寫入時一律以 $\min(i,j)$ 為列、$\max(i,j)$ 為欄：

  $$tabu_list[\min(i,j)][\max(i,j)] =$$

  $$current_iteration + tabu_tenure$$

  在後續迭代中，若某個交換動作對應的陣列值大於當前代數，則視為禁忌狀態。
- **Method 2:** 基於排列狀態的禁忌記錄 **(Permutation Hash-based Tabu List)**
  在 `tabu_search2` 函數中，我們將禁忌對象從「交換動作」改為「整體的排列狀態」。為了高效比對狀態，實作中採用了多項式滾動雜湊（Polynomial Rolling Hash）：

  $$H = \sum_{k=0}^{n-1} (arr[k] + 1) \cdot p^k \pmod q$$

  其中 $arr[k]$ 為排列中第 k 個位置的工作 ID，質數 p 設為 313。模數 q 則巧妙利用 C++ 中 `uint64_t` 型態的自然溢位特性，等同於 $q = 2^{64}$。每次步入新的排列狀態後，會將其 Hash 值存入 `std::unordered_map` 並記錄解禁代數。此方法能更嚴格地避免走訪重複的解空間。

特赦條件 **(Aspiration Criterion)**
無論使用何種記錄方式，若產生的相鄰解其 Makespan 嚴格小於歷史最佳解（Global Best），則觸發特赦機制，演算法將無視禁忌狀態強制接受該解。

E. Gnuplot Visualizer

為了直觀分析演算法的收斂行為與跳脫區域最佳解的過程，本實作內建了基於 `Gnuplot` 的視覺化工具。程式透過 `popen` 開啟 `gnuplot` 的行程通訊（Pipe），並利用標準輸入字元 `'-'` 將 C++ 陣列中的 `best_makespan_history` 與 `makespan_history` 即時傳遞給繪圖引擎。

在大量平行測試的環境下，系統會透過 `save_plot` 函數在背景靜默生成圖表。該函數會自動切換 Gnuplot 的 `terminal` 為 `jpeg`，並根據演算法名稱、測資名稱及最佳解數值自動命名檔案存入指定目錄，確保實驗數據的可重現性與易讀性。

V. Experiment

本研究針對 Taillard 測試案例進行實驗，每個案例重複執行 20 次以確保統計穩定性。

A. Different Algorithm

如表 I 所示，比較 II、SA 與 TS 三種演算法。實驗結果顯示，**TS (Tabu Search)** 在所有測試案例中均獲得最低的完工時間 (Makespan)，特別是在大型測資 (如 tai100 系列) 中，其平均值 (Avg) 顯著優於 II 與 SA。本實驗結果顯示禁忌搜尋透過記憶機制能更有效地引導搜尋過程離開區域最佳解。

TABLE I
不同演算法之結果比較

Instance	II			SA			TS		
	Min	Avg	Max	Min	Avg	Max	Min	Avg	Max
tai20_5_1	1297	1331.5	1377	1297	1305.2	1358	**1278**	**1297.8**	1324
tai20_10_1	1624	1676.8	1770	1607	1654.0	1686	**1582**	**1587.7**	1597
tai100_5_1	5527	5559.3	5628	5512	5558.3	5626	**5495**	**5513.7**	5529
tai100_10_1	6004	6107.5	6230	5995	6064.3	6127	**5796**	**5845.1**	5912

B. Different Cooling Parameter for SA

針對模擬退火法，我們測試了初始溫度 T 與冷卻率 α 的組合。由表 II 可見，較高的初始溫度 ($T = 1000$) 配合較慢的降溫率 ($\alpha = 0.95$)，雖然計算開銷較大，但在大型測資 (如 tai100_20_1) 中能獲得較佳的 Min 與 Avg。快速降溫 ($\alpha = 0.8$) 雖然收斂快，但容易過早陷入區域最佳解。

TABLE II
SA 不同參數組合對結果之影響

Instance	$T = 1000, \alpha = 0.95$			$T = 100, \alpha = 0.95$			$T = 1000, \alpha = 0.8$		
	Min	Avg	Max	Min	Avg	Max	Min	Avg	Max
tai100_5_1	5507	5547.3	5573	5500	5556.7	5616	5500	5555.5	5613
tai100_10_1	5981	6090.4	6166	5986	6084.2	6192	5983	6076.9	6165
tai100_20_1	**6732**	**6842.4**	6976	6744	6854.2	6927	6769	6869.2	7024

C. Different Length of Tabu Tenure for TS

本實驗進一步分析禁忌長度（Tabu Tenure, L）對搜尋表現的影響。如表 III 所示，較長的禁忌期（$L = 15$ 與 $L = 10$）在多數測資中的整體表現（特別是平均完工時間）明顯優於較短的禁忌期（$L = 5$）。

分析原因在於，當 $L = 5$ 時禁忌名單過短，導致演算法容易在少數幾個區域最佳解之間發生循環（Cycling），缺乏足夠的多樣化（Diversification）能力來探索未知的解空間。反之，適當增加禁忌長度能有效強迫搜尋軌跡遠離已造訪過的結構；其中 $L = 10$ 在部分小型測資（如 tai20 系列）能靈活找到更低的最小值（Min），而 $L = 15$ 則在大型測資（如 tai100 系列）中展現出更強的探索能力與更穩定的平均收斂品質。

TABLE III
TS 不同禁忌長度對結果之影響

Instance	$L = 15$			$L = 10$			$L = 5$		
	Min	Avg	Max	Min	Avg	Max	Min	Avg	Max
tai100_10_1	5800	5840.80	5912	**5799**	5849.45	5912	5828	5898.80	5939
tai100_20_1	**6386**	6439.70	6514	6413	6453.15	6501	6415	6490.85	6582
tai100_5_1	**5495**	5517.30	5546	**5495**	5517.75	5564	**5495**	5520.40	5546
tai20_10_1	1583	1589.00	1595	**1582**	1589.50	1599	1583	1603.30	1653
tai20_20_1	2303	2324.40	2357	**2302**	2313.15	2346	**2302**	2345.05	2392
tai20_5_1	1283	1296.95	1305	**1278**	1305.25	1339	**1278**	1309.30	1339
tai50_10_1	**3046**	3082.00	3135	3048	3083.05	3131	**3046**	3113.35	3160
tai50_20_1	**3920**	3942.35	4045	3921	3949.10	4010	3934	3988.75	4055
tai50_5_1	**2724**	2743.80	2758	**2724**	2745.70	2774	**2724**	2744.85	2774

D. Different Storing Objective for TS

最後比較移動禁忌 (Move-based) 與狀態雜湊禁忌 (State-based Hash)。如表 IV 所示，移動禁忌在多數情況下的平均完工時間與搜尋穩定性均優於狀態雜湊禁忌。如表 IV 所示，移動禁忌在多數情況下的平均完工時間均優於雜湊禁忌。分析原因在於，雜湊禁忌僅能防止回到「完全相同」的解，對於鄰域中極其相似的解空間缺乏約束力，而移動禁忌則能強制搜尋跳脫目前的結構區域。

TABLE IV
MOVE-BASED 與 STATE-BASED HASH 之效能比較

Instance	**TS (Move-based)**			**TS2 (State-based)**		
	Min	Avg	Max	Min	Avg	Max
tai20_5_1	**1278**	**1306.00**	1339	1297	1326.15	1377
tai20_10_1	**1582**	**1586.90**	1596	1586	1622.85	1671
tai20_20_1	**2298**	**2318.65**	2353	2309	2361.65	2435
tai50_5_1	**2724**	**2725.25**	2735	2724	2730.05	2761
tai50_10_1	**3053**	**3093.15**	3151	3081	3129.70	3167
tai50_20_1	**3916**	**3941.40**	3971	3943	4038.50	4124
tai100_5_1	**5495**	**5512.10**	5564	5495	5518.45	5564
tai100_10_1	**5785**	**5831.70**	5912	5800	5895.85	5969
tai100_20_1	**6394**	**6449.60**	6528	6453	6540.45	6614

VI. CONCLUSION

REFERENCES

REFERENCES

[1] M. Young, The Technical Writer's Handbook. Mill Valley, CA: University Science, 1989.